HALO Platform Expansion — Detailed Product Requirements Document (v2.1)

Product: HALO (Hallucination-Aware Legal Output) — *Nyaya Sahayak* **Scope of this document:** Everything *beyond* the verified RAG core: Drafting Studio, Practice Dashboard & Calendar, Telegram Gateway, and the platform foundations they need. **Status date:** 4 November 2026

> ***Scope note — RAG core.*** *The RAG bot / verification engine (`backend/halo/`, retrieval, generation, audit) is* ***COMPLETE*** *and is treated here as a stable, versioned dependency.* ***Integration of the new modules with the RAG core will be done afterwards****, once the modules in this document are built. Where a feature needs RAG output (e.g. clause verification), this PRD defines only the integration seam (an interface + a stub), not the RAG internals.*

Table of Contents

1. Executive Summary
2. Current State Baseline
3. Goals, Non-Goals, Personas
4. Target Architecture
5. Roadmap & Phasing
6. Phase 2 — Document Drafting Engine
7. Phase 3 — Practice Dashboard & Case Calendar
8. Phase 4 — Telegram Bot Gateway
9. Cross-Cutting: Auth, Tenancy, Security
10. Data Model
11. API Specification
12. Non-Functional Requirements
13. RAG Integration Seam (Deferred)
14. Testing & Quality Strategy
15. Risks, Open Questions & Decisions
16. Definition of Done & Milestones
17. Appendix

---

1. Executive Summary

HALO started as an evidence-verified legal research engine. This expansion turns it into an **end-to-end practice management ecosystem for Indian legal practitioners**, adding:

Phase	Module	Outcome for the lawyer
2	**Drafting Engine**	Pick a statutory template → fill a schema-driven form → get a formatted `.docx` / `.pdf` draft
3	**Practice Dashboard & Calendar**	One place for matters, clients, hearings, and auto-computed statutory deadlines
4	**Telegram Gateway**	Query, check agenda, and receive hearing alerts from a phone

The RAG core is done and will be wired in after these modules exist, so every module below is designed to **work standalone first** and **accept verification later** through a defined seam (§13).

---

2. Current State Baseline

2.1 What exists (and is out of scope for rebuilding)

Area	State
RAG / Verification core	Done. 8-stage pipeline: claim extractor → 3-tier citation verifier → evidence/NLI verifier → temporal verifier → conflict detector → confidence engine → fail-closed governor → SHA-256 audit logger
Retrieval	BM25 + BGE dense + RRF (k=60) over statutory & judicial corpora
Backend foundation	FastAPI 0.115+, Python 3.12, SQLModel (SQLite), JWT (HS256), Passlib/Bcrypt, 4 roles (`LAWYER`, `RESEARCHER`, `ADMIN`, `INSTITUTION_ADMIN`)
Frontend	React 19 + Vite + React Router 7, 11 routed pages, `--halo-*` design tokens, 0 build errors
Corpora	Companies Act 2013 (1,640 passages), 57 judgments (1,133 passages), 1,032 benchmark queries (offline only)
Docs	PRD v2.0 (77 sections), PROJECT_STATUS, mid-sem guide, research paper drafts

2.2 Existing assets the new modules should reuse

- **Auth/RBAC** (`backend/app/auth/`) — extend, don't replace.
- **Drafting controller** (`backend/app/generation/controller.py`) — already generates three pleadings (Affidavit of Evidence, Director Due-Diligence Affidavit, Notice under S.138 NI Act). Treat as a **prototype to be absorbed** into the schema-driven engine.
- **Frontend pages** `DraftingPage.jsx` and `CasesPage.jsx` — present as screens; they are the starting point for Phase 2 and Phase 3 UI.
- **AppShell / design system** — all new screens must use it.
- **Audit ledger** — new modules emit events into the same append-only pattern where meaningful.

2.3 Known gaps to close before/while building

#	Gap	Impact	Action
G1	Backend pytest suite exits with code 1	No green baseline	Triage and fix first (Sprint 0)
G2	SQLite only	Multi-tenancy + concurrency + scheduled jobs need Postgres	Migrate to PostgreSQL (Sprint 0)
G3	Drafting is hard-coded to 3 documents	Not extensible	Replace with template registry (Phase 2)
G4	`CasesPage` is tied to research queries, not real matters	No client/matter model	Introduce Matter domain (Phase 3)
G5	No background job runner	Needed for reminders, deadline calc, async export	Add task queue (Sprint 0)
G6	Repo layout is a monorepo (`backend/`, `frontend/`), PRD assumed two repos	Ambiguity	See Decision D1 (§15)

---

3. Goals, Non-Goals, Personas

3.1 Goals

1. Reduce time to produce a standard statutory draft from hours to minutes.
2. Give a lawyer a single source of truth for matters, hearings, and deadlines.
3. Make deadlines **computed, explainable, and never silently wrong**.
4. Offer secure mobile access for read-mostly workflows.
5. Keep every module integration-ready for the (completed) RAG core.

3.2 Non-Goals (this document)

- Rebuilding or modifying the RAG/verification engine.
- Billing, invoicing, time tracking, e-filing portal automation (NCLT/e-Courts filing).
- Non-Companies-Act practice areas beyond the templates listed.
- Native mobile apps.
- Client-facing portal.

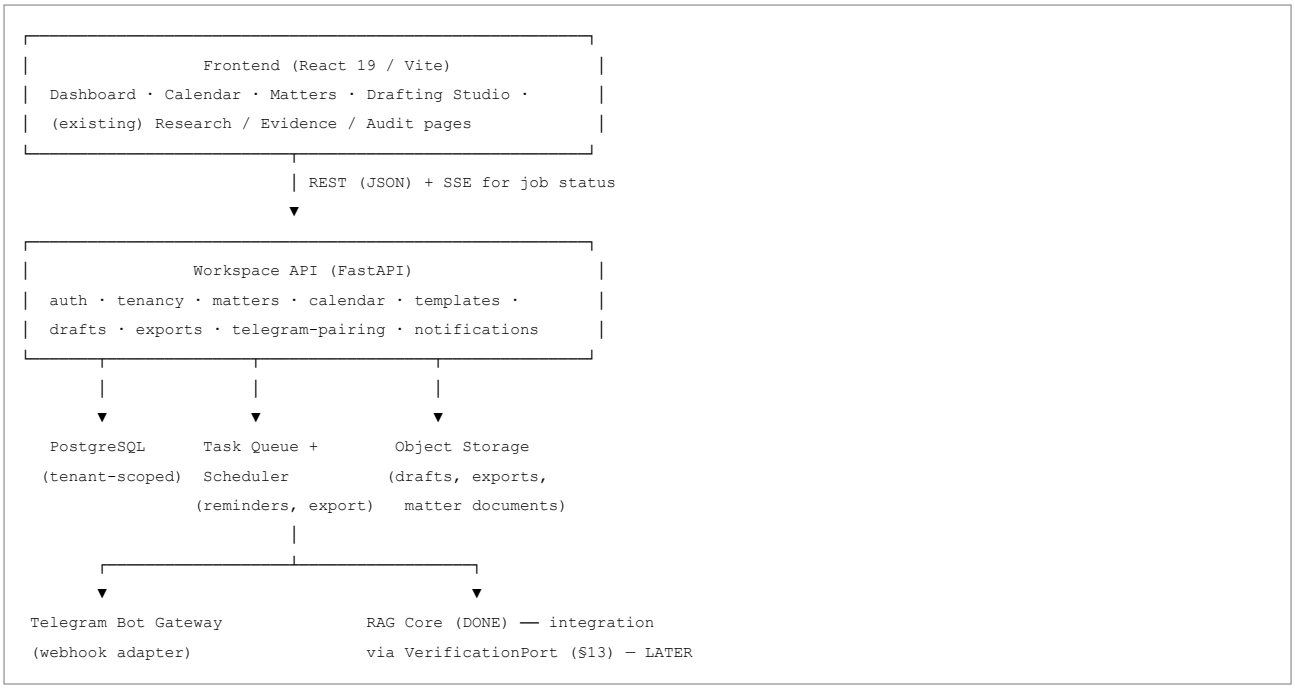
3.3 Personas

Persona	Needs
Advocate (primary)	Fast drafting, never miss a date, mobile glance at agenda
Junior / Associate	Fill forms, prepare drafts, scoped to firm matters
Firm Admin	Manage users, templates, firm-wide calendar
Researcher	Existing research flows (unchanged)

---

4. Target Architecture

4.1 Logical view



4.2 Key architectural principles

1. **Ports & adapters.** The RAG core is reached only through a `VerificationPort` / `ResearchPort` interface. Phase 2–4 ship with a **no-op/stub adapter**; the real adapter is plugged in during integration.
2. **Tenant-first.** Every table carries `firm_id`; every query is tenant-filtered at the repository layer (not the controller).
3. **Deterministic over generative for dates.** Statutory deadlines come from a **rules engine**, not an LLM.
4. **Async for anything > 500 ms.** Export, bulk deadline recompute, and reminder fan-out run as jobs.
5. **Separation of production vs. evaluation paths** (carried over from the core design): nothing in these modules touches the quarantined benchmark dataset.

4.3 Deployment topology (initial)

- Single FastAPI service (API + worker as separate processes), PostgreSQL, Redis (queue/broker), S3-compatible storage (MinIO locally).
- Telegram webhook served by the same API under `/telegram/webhook` with secret-token header validation.

---

5. Roadmap & Phasing

```
Sprint 0  Foundations        ── fix tests, Postgres, tenancy, queue, storage
Phase 2   Drafting Engine    ── template registry, forms, assembly, export   ← CURRENT
Phase 3   Dashboard/Calendar ── matters, hearings, deadline rules engine     ← NEXT
Phase 4   Telegram Gateway   ── pairing, commands, reminders                 ← FINAL
Phase 5   RAG Integration    ── plug real adapters into the seams            ← AFTERWARDS
```

5.1 Dependency graph

- Phase 2 needs: Sprint 0 (storage, queue).
- Phase 3 needs: Sprint 0 (Postgres, scheduler); benefits from Phase 2 (drafts attach to matters).
- Phase 4 needs: Phase 3 (matters + calendar data) and Sprint 0 (queue).
- Phase 5 needs: Phases 2–4 complete.

---

6. Phase 2 — Document Drafting Engine

6.1 Overview

Lawyers choose a **statutory template**, complete a **schema-driven form**, and receive a **fully formatted draft** exportable to `.docx` and `.pdf`.

6.2 Functional requirements

ID	Requirement	Priority

ID	Requirement	Priority
DR-01	Template registry: list, view, version templates, scoped global + per-firm	P0
DR-02	Each template declares a JSON **form schema** (fields, types, validation, conditional visibility)	P0
DR-03	Frontend renders forms dynamically from schema (no per-template UI code)	P0
DR-04	Server-side validation mirrors schema (never trust client)	P0
DR-05	Assembly via Jinja2 + `python-docx` with clause injection	P0
DR-06	Export to editable `.docx`	P0
DR-07	Export to `.pdf` (non-editable; document hash printed in footer)	P0
DR-08	Drafts persist, are versioned, and can be reopened/edited	P0
DR-09	Draft can be attached to a Matter (Phase 3)	P1
DR-10	Draft status lifecycle: `DRAFT → GENERATING → READY → FINALISED`	P1
DR-11	Optional **verification hook** per clause (stubbed until integration)	P1
DR-12	Firm admins can upload/customise templates	P2
DR-13	Preview before export (rendered HTML)	P1
DR-14	Migrate the 3 existing pleadings into the template registry	P0

6.3 Template package format

Each template is a directory/record:

```
templates/BOARD_RES_DIR_APPT_V1/
├── manifest.json        # id, title, version, statutory_anchor, category
├── schema.json          # form fields + validation
├── body.docx.j2         # Jinja2-enabled .docx (or body.j2 + style map)
├── clauses/             # optional reusable clause blocks
└── tests/fixtures.json  # sample inputs + expected output hash
```

manifest.json

```
{
  "template_id": "BOARD_RES_DIR_APPT_V1",
  "title": "Board Resolution — Appointment of Director (Additional)",
  "version": "1.0.0",
  "statutory_anchor": "Section 161(1), Companies Act, 2013",
  "category": "CORPORATE_RESOLUTION",
  "output_formats": ["docx", "pdf"],
  "verification": { "enabled": false, "mode": "clause" }
}
```

6.4 Form schema specification

Supported field types: `string`, `text`, `number`, `date`, `select`, `multiselect`, `boolean`, `party`, `repeatable_group`.

Validation primitives: `required`, `pattern`, `min/max`, `min_length/max_length`, `enum`, `custom` (named validators).

Built-in Indian-law validators | Validator | Rule | |---|---| | `cin` | 21-char CIN format (listing status, state, year, ownership, number) | | `din` | 8-digit DIN | | `pan` | `AAAAA9999A` | | `gstin` | 15-char GSTIN with checksum | | `date_not_future` / `date_after(field)` | Cross-field date logic |

Example

```
{
  "template_id": "BOARD_RES_DIR_APPT_V1",
  "statutory_anchor": "Section 161(1), Companies Act, 2013",
  "fields": [
    { "id": "company_cin",    "label": "CIN",                        "type": "string", "required": true, "validator": "cin" },
    { "id": "company_name",   "label": "Company Name",               "type": "string", "required": true },
    { "id": "director_din",   "label": "Appointee DIN",              "type": "string", "required": true, "validator": "din" },
    { "id": "director_name",  "label": "Appointee Name",             "type": "string", "required": true },
    { "id": "effective_date", "label": "Effective Date",             "type": "date",   "required": true },
    { "id": "meeting_date",   "label": "Board Meeting Date",         "type": "date",   "required": true, "validator": "date_not_futu
    { "id": "articles_authorise", "label": "Articles authorise additional directors", "type": "boolean", "required": true },
    { "id": "co_signatory",   "label": "Co-signatory (Chairman/Director)", "type": "party", "required": true }
  ],
  "conditional": [
    { "if": { "articles_authorise": false },
      "then": { "warn": "Section 161(1) requires authorisation in the Articles." } }
  ]
}
```

6.5 Initial template catalogue

Template ID	Title	Anchor	Priority
`BOARD_RES_DIR_APPT_V1`	Board Resolution – Appointment of Additional Director	S.161(1) CA 2013	P0
`PETITION_OPPR_MISMGMT_V1`	Petition – Oppression & Mismanagement	S.241/242 CA 2013	P0
`AFFIDAVIT_EVIDENCE_V1`	Affidavit of Evidence	Order XIX R.3 CPC	P0 (migrated)
`AFFIDAVIT_DIR_DD_V1`	Director Due-Diligence Affidavit	S.179 IT Act	P0 (migrated)
`NOTICE_DEMAND_138_V1`	Statutory Demand Notice	S.138 NI Act	P0 (migrated)
`BOARD_RES_GENERIC_V1`	Generic Board Resolution	—	P1
`MGT14_COVER_V1`	MGT-14 filing cover note	S.117 CA 2013	P2

6.6 Assembly pipeline

```
Form input → Validate (schema + validators)
          → Normalise (dates → DD Month YYYY, names → title case rules, numbers → words)
          → Render (Jinja2 context → .docx via python-docx)
          → [Verification hook — stub now, RAG later]
          → Persist DraftVersion (+ content hash)
          → Export job (docx / pdf)  → Object storage → signed download URL
```

Rendering rules

- Styles come from a per-template style map (fonts, margins, numbering) so output is court-presentable.
- Numbered clauses use real Word numbering, not typed numbers.
- Placeholders never render blank silently: a missing optional value is removed with its sentence; a missing required value aborts with a field-level error.
- Rupee amounts rendered in Indian numbering (₹ `12,50,000`) plus words.

PDF export: render from `.docx` via LibreOffice headless (primary) with a fallback path; embed SHA-256 of the final content in the footer; set PDF permissions to disallow editing.

6.7 UX requirements (Drafting Studio)

1. **Template gallery** with category filters and search.
2. **Stepper form**: sections → review → generate.
3. Inline validation with plain-language errors.
4. **Autosave** every 10 s to a draft record.
5. **Live preview** pane (HTML render of current inputs).
6. Version history with diff between versions.
7. Export menu: `.docx`, `.pdf`; copy link (internal).
8. Disabled state with tooltip where verification is "pending integration".

6.8 Acceptance criteria

- A user can generate `BOARD_RES_DIR_APPT_V1` end-to-end and download a correct `.docx` and `.pdf`.
- Invalid CIN/DIN are rejected client- and server-side with specific messages.
- Assembly + PDF export for a one-page resolution completes in **< 1.5 s** (p95).
- All 3 legacy pleadings produce output equivalent to current behaviour (golden-file tests).

- Adding a new template requires **zero frontend code changes**.

---

7. Phase 3 — Practice Dashboard & Case Calendar

7.1 Overview

A web dashboard for open matters, filings, hearings, and **auto-computed statutory deadlines**.

7.2 Functional requirements

Matter & client management

ID	Requirement	Priority
MT-01	CRUD for Clients (individual / company; CIN, PAN optional)	P0
MT-02	CRUD for Matters tagged with client, CIN, parties, forum, case no., lead advocate, status	P0
MT-03	Forum enum: NCLT (by bench), NCLAT, High Court (by court), Supreme Court, Other	P0
MT-04	Matter team: lead advocate + assigned juniors with roles	P1
MT-05	Matter documents: upload, tag, preview (PDF viewer), link drafts from Phase 2	P0
MT-06	Matter notes (timestamped, author-attributed, editable with history)	P1
MT-07	Matter timeline (hearings, filings, drafts, notes in one feed)	P1
MT-08	Search & filter (status, forum, client, advocate, date)	P0
MT-09	Link research queries (existing history) to a matter	P1 (via integration)

Calendar & deadlines

ID	Requirement	Priority
CL-01	Month / week / day / agenda views	P0
CL-02	Event types: `HEARING`, `FILING_DEADLINE`, `CLIENT_MEETING`, `COMPLIANCE`, `OTHER`	P0
CL-03	Hearing priority colour coding (SC > HC/NCLAT > NCLT > other)	P1
CL-04	**Deadline rules engine** (see §7.4)	P0
CL-05	Recurring events and next-hearing-date updates after each hearing	P1
CL-06	Reminders: configurable offsets (default 24 h and 2 h)	P0
CL-07	iCal export / subscribe feed (tokenised URL)	P2
CL-08	Conflict detection (two hearings overlapping for the same advocate)	P2
CL-09	Holiday awareness (court calendars / vacations) as a configurable dataset	P2

Dashboard widgets

- Today's agenda · Upcoming 7-day hearings · Overdue/at-risk deadlines · Active matters by forum · Drafts in progress · Recent activity.

7.3 Matter lifecycle

```
INTAKE → ACTIVE → (STAYED | RESERVED_FOR_ORDERS) → DISPOSED → ARCHIVED
```

Status transitions are logged with actor + timestamp.

7.4 Statutory deadline rules engine

Principle: deadlines are produced by **declarative, versioned rules** with a human-readable explanation, never opaque.

Rule shape

```
{
  "rule_id": "MGT14_FILING_30D",
  "title": "File MGT-14 within 30 days of passing the resolution",
  "trigger_event": "BOARD_RESOLUTION_PASSED",
  "anchor_date_field": "resolution_date",
  "offset": { "days": 30, "type": "calendar" },
  "statutory_basis": "Section 117(1), Companies Act, 2013",
  "extension": { "additional_fee_window_days": 270, "note": "Late filing attracts additional fees" },
  "rule_version": "2026.1"
}
```

Behaviour

- Trigger events raise **proposed** `FILING_DEADLINE` calendar events; user confirms or edits.

- Every computed deadline stores `rule_id`, `rule_version`, inputs, and an **explanation string** ("Resolution on 3 Nov 2026 + 30 days = 3 Dec 2026, per S.117(1)").
- Handles calendar vs. working days, and rolls forward when a due date falls on a holiday **only where the rule says so**.
- Rules are data, reviewed by a lawyer, and covered by golden tests.
- **Disclaimer in UI:** computed dates are aids; the advocate is responsible for verification.

Seed rules (P0) | Rule | Basis | |---|---| | MGT-14 within 30 days of resolution | S.117 CA 2013 | | DIR-12 within 30 days of director appointment | S.170 / Rule 18 | | AOC-4 within 30 days of AGM | S.137 | | MGT-7 within 60 days of AGM | S.92 | | Appeal to NCLAT within 45 days of NCLT order (+ condonable 45) | S.421(3) | | Notice under S.138 NI Act: 30 days from dishonour intimation; complaint within 1 month after 15-day payment period | NI Act S.138 |

> *Every rule above must be **verified by a practising lawyer before release** (see Risk R2).*

7.5 Acceptance criteria

- Creating a matter, adding a hearing, and seeing it on calendar/agenda works end-to-end for two tenants without cross-visibility.
- Recording a resolution date generates the correct proposed MGT-14 deadline with explanation.
- Reminder fires within ±60 s of scheduled time.
- Dashboard loads in **< 2 s** for a firm with 5,000 matters / 50,000 events.

---

8. Phase 4 — Telegram Bot Gateway

8.1 Overview

A light, secure interface for lawyers on the move. Read-mostly; no destructive actions.

8.2 Pairing & authentication

1. User opens **Settings** → **Telegram** in web app → clicks *Generate pairing code*.
2. Server issues a **6-digit OTP**, valid **5 min**, single-use, rate-limited (5/hour/user).
3. User sends `/auth 123456` to the bot.
4. Server binds `telegram_user_id` ↔ `user_id` and returns confirmation. Subsequent updates authenticate by the bound ID.
5. User may **unpair** from the web app or `/logout` in the bot; admins may revoke.

Security details

- Webhook protected by `X-Telegram-Bot-Api-Secret-Token`.
- Reject updates from unpaired IDs except `/start`, `/help`, `/auth`.
- Bot tokens/secrets only in environment, never in DB or logs.
- Sensitive output (matter details) is sent only in **private chats**, never groups.
- Optional **session timeout** (re-auth after N days, default 30).

8.3 Command specification

Command	Parameters	Function	Output	Phase dependency
`/start`, `/help`	—	Intro, command list	Text	none
`/auth`	`<otp>`	Pair account	Status confirmation	Sprint 0
`/logout`	—	Unpair	Confirmation	Sprint 0
`/cases`	`[search]`	Top 5 active matters	Inline keyboard menu	Phase 3
`/case`	`<name or id>`	Matter summary: status, next hearing, latest note	Card message	Phase 3
`/agenda`	`today | tomorrow | week`	Hearings & deadlines	Formatted list	Phase 3
`/draft_status`	`<draft_id>`	Status of async generation	Status + download link	Phase 2
`/ask`	`<legal query>`	Verified RAG answer	Answer + citation badges	**Phase 5 (integration)**

> `/ask` *is **specified now but wired later**. Until integration, it replies: "Legal research via Telegram is coming soon."*

8.4 Notifications

- Hearing reminders at **24 h** and **2 h** before (per-user configurable).
- Deadline reminders at **7 d / 1 d / day-of**.
- Morning digest (optional, default 08:00 IST): today's agenda.
- Delivery tracking: `notification_log` with status; retries with backoff; user-blocked-bot handled gracefully (auto-pause).

- De-duplication using `reminder_sent` flags keyed by (event, offset).

8.5 Message design

- Telegram MarkdownV2 with strict escaping utility.
- Max message length handled via pagination ("More ▸" inline button).
- Download links are **short-lived signed URLs** (≤ 10 min).
- Time rendered in the user's timezone (default `Asia/Kolkata`).

8.6 Acceptance criteria

- Pairing works with OTP; expired/reused OTP rejected.
- Unpaired user cannot read any matter data.
- `/agenda today` responds in **< 3.0 s** (p95).
- A hearing reminder is delivered at T-24 h and T-2 h exactly once.

---

9. Cross-Cutting: Auth, Tenancy, Security

9.1 Identity & access

- Extend existing JWT auth: access token (15 min) + refresh token (7 d, rotating), stored httpOnly.
- Optional **TOTP MFA** for dashboard login (P1).
- Roles → permissions matrix:

Capability	LAWYER	JUNIOR	RESEARCHER	ADMIN / FIRM ADMIN
Create/edit own matters	□	□ (assigned)	□	□
View firm matters	assigned	assigned	□	□
Generate drafts	□	□	□	□
Manage templates	□	□	□	□
Pair Telegram	□	□	□	□
Manage users	□	□	□	□

9.2 Multi-tenancy

- `firm_id` on all tenant tables; enforced by a repository base class that injects the filter.
- PostgreSQL **Row-Level Security** as defence-in-depth (P1).
- Per-tenant object-storage prefix; signed URLs only.
- Cross-tenant access tests are **mandatory** in CI.

9.3 Data protection (India context)

- Encryption in transit (TLS) and at rest (DB + storage).
- Advocate–client confidentiality: no matter content in logs, analytics, or error trackers.
- DPDP Act 2023 alignment: purpose limitation, deletion/export on request, retention policy per matter.
- Audit events for: login, pairing, draft export, matter access, deletion.

9.4 Secrets & config

- `.env` per service with a committed `.env.example`; no secrets in repo.
- Separate configs for dev / staging / prod.

---

10. Data Model

10.1 Entity overview

```
Firm 1—* User
Firm 1—* Client 1—* Matter 1—* CalendarEvent
                       |  └—* MatterDocument
                       |  └—* MatterNote
                       └—* MatterMember *—1 User
Firm 1—* Template 1—* TemplateVersion
Matter 0..1—* Draft 1—* DraftVersion 1—* Export
User 1—* TelegramLink
CalendarEvent 1—* Reminder 1—* NotificationLog
DeadlineRule *—* CalendarEvent (via computation record)
```

10.2 Improved DDL (PostgreSQL)

> *Improves the blueprint in the original PRD: adds tenancy, clients, forum structure, soft delete, audit columns, and indexes.*

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE firms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE clients (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firm_id UUID NOT NULL REFERENCES firms(id),
  client_name VARCHAR(255) NOT NULL,
  client_type VARCHAR(20) NOT NULL CHECK (client_type IN ('INDIVIDUAL','COMPANY','LLP','OTHER')),
  cin VARCHAR(21),
  pan VARCHAR(10),
  email VARCHAR(255),
  phone VARCHAR(32),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);
CREATE INDEX idx_clients_firm ON clients(firm_id) WHERE deleted_at IS NULL;

CREATE TABLE legal_matters (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firm_id UUID NOT NULL REFERENCES firms(id),
  client_id UUID NOT NULL REFERENCES clients(id),
  matter_name VARCHAR(255) NOT NULL,
  forum_type VARCHAR(30) NOT NULL,         -- NCLT | NCLAT | HIGH_COURT | SUPREME_COURT | OTHER
  forum_detail VARCHAR(120),               -- e.g. 'Mumbai Bench'
  case_number VARCHAR(100),
  opposing_parties TEXT,
  statutory_act VARCHAR(100) DEFAULT 'Companies Act, 2013',
  lead_advocate_id UUID REFERENCES users(id),
  status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);
CREATE INDEX idx_matters_firm_status ON legal_matters(firm_id, status);
CREATE INDEX idx_matters_name_trgm ON legal_matters USING gin (matter_name gin_trgm_ops);

CREATE TABLE calendar_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firm_id UUID NOT NULL REFERENCES firms(id),
  matter_id UUID REFERENCES legal_matters(id) ON DELETE CASCADE,
  event_title VARCHAR(255) NOT NULL,
  event_type VARCHAR(30) NOT NULL,         -- HEARING | FILING_DEADLINE | CLIENT_MEETING | COMPLIANCE | OTHER
  starts_at TIMESTAMPTZ NOT NULL,
  ends_at TIMESTAMPTZ,
  all_day BOOLEAN NOT NULL DEFAULT FALSE,
  status VARCHAR(20) NOT NULL DEFAULT 'CONFIRMED',   -- PROPOSED | CONFIRMED | DONE | CANCELLED
  priority SMALLINT NOT NULL DEFAULT 3,
  rule_id VARCHAR(80),
  rule_version VARCHAR(20),
  computation_explanation TEXT,
  notes TEXT,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_events_firm_time ON calendar_events(firm_id, starts_at);
CREATE INDEX idx_events_matter ON calendar_events(matter_id);

CREATE TABLE reminders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  event_id UUID NOT NULL REFERENCES calendar_events(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id),
  offset_minutes INTEGER NOT NULL,
  channel VARCHAR(20) NOT NULL,            -- TELEGRAM | EMAIL | IN_APP
  fire_at TIMESTAMPTZ NOT NULL,
  sent_at TIMESTAMPTZ,
  UNIQUE (event_id, user_id, offset_minutes, channel)
);
CREATE INDEX idx_reminders_due ON reminders(fire_at) WHERE sent_at IS NULL;

CREATE TABLE templates (
  id VARCHAR(80) PRIMARY KEY,              -- e.g. BOARD_RES_DIR_APPT_V1
  firm_id UUID REFERENCES firms(id),       -- NULL = global
  title VARCHAR(255) NOT NULL,
  category VARCHAR(60) NOT NULL,
  statutory_anchor VARCHAR(255),
  active_version VARCHAR(20) NOT NULL
);

CREATE TABLE template_versions (
  template_id VARCHAR(80) REFERENCES templates(id),
  version VARCHAR(20),
  schema_json JSONB NOT NULL,
  body_ref TEXT NOT NULL,                  -- storage key to .docx.j2
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (template_id, version)
);

CREATE TABLE drafts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  firm_id UUID NOT NULL REFERENCES firms(id),
  matter_id UUID REFERENCES legal_matters(id),
  template_id VARCHAR(80) NOT NULL,
  title VARCHAR(255) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE draft_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  draft_id UUID NOT NULL REFERENCES drafts(id) ON DELETE CASCADE,
  version_no INTEGER NOT NULL,
  input_json JSONB NOT NULL,
  content_sha256 CHAR(64),
  verification_report JSONB,               -- NULL until integration
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (draft_id, version_no)
);

CREATE TABLE exports (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  draft_version_id UUID NOT NULL REFERENCES draft_versions(id),
  format VARCHAR(10) NOT NULL,             -- docx | pdf
  storage_key TEXT,
  status VARCHAR(20) NOT NULL DEFAULT 'QUEUED',
  error TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE telegram_links (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  telegram_user_id BIGINT NOT NULL UNIQUE,
  paired_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  revoked_at TIMESTAMPTZ
```

```
);

CREATE TABLE telegram_pairing_codes (
  code_hash CHAR(64) PRIMARY KEY,          -- store hash, not the OTP
  user_id UUID NOT NULL REFERENCES users(id),
  expires_at TIMESTAMPTZ NOT NULL,
  used_at TIMESTAMPTZ
);
```

---

11. API Specification

Base: `/api/v1`. All endpoints require auth unless stated; all are tenant-scoped.

11.1 Templates & Drafts (Phase 2)

Method	Path	Purpose
GET	`/templates`	List templates (filter by category)
GET	`/templates/{id}`	Manifest + active schema
POST	`/drafts`	Create draft from template + inputs
GET	`/drafts/{id}`	Draft + versions + status
PUT	`/drafts/{id}/inputs`	Autosave inputs (creates/updates working version)
POST	`/drafts/{id}/generate`	Validate + assemble (async)
GET	`/drafts/{id}/preview`	HTML preview
POST	`/drafts/{id}/exports`	Request export `{format}`
GET	`/exports/{id}`	Status + signed URL

11.2 Matters & Calendar (Phase 3)

Method	Path	Purpose
CRUD	`/clients`, `/matters`	Core entities
POST	`/matters/{id}/documents`	Upload
GET	`/matters/{id}/timeline`	Merged activity feed
CRUD	`/events`	Calendar events
GET	`/events?from=&to=&matter_id=&type=`	Range query
POST	`/deadlines/compute`	`{rule_id, anchor_date, matter_id}` → proposed event + explanation
POST	`/events/{id}/confirm`	Confirm proposed deadline
GET	`/dashboard/summary`	Widget data
GET	`/calendar.ics?token=`	iCal feed

11.3 Telegram (Phase 4)

Method	Path	Purpose
POST	`/telegram/pairing-codes`	Generate OTP (web, authed)
DELETE	`/telegram/link`	Unpair
POST	`/telegram/webhook`	Bot updates (secret-token auth, no JWT)

11.4 Conventions

- JSON errors: `{ "error": { "code", "message", "fields": [...] } }`.
- Pagination: `?limit=&cursor=`.
- Idempotency-Key header on POSTs that enqueue jobs.
- OpenAPI schema auto-published; frontend client generated from it.

---

12. Non-Functional Requirements

Category	Requirement	Target
Performance	Document assembly + PDF export (single page)	< 1.5 s p95
	Telegram command response (non-RAG)	< 3.0 s p95
	Dashboard load	< 2 s p95
	Calendar range query (1 month)	< 300 ms p95
	RAG query with verification (**post-integration**)	< 4.5 s p95
Reliability	API availability	99.5 %
	Reminder delivery success	≥ 99 % within ±60 s

Category	Requirement	Target
	Job retry policy	3 retries, exponential backoff, dead-letter queue
Security	OWASP ASVS L2 baseline; dependency scanning in CI	Required
Privacy	Zero matter content in logs	Required
Scalability	100 firms, 5k matters/firm, 50 concurrent users/firm	Initial target
Observability	Structured logs, request IDs, metrics (latency, job lag, reminder lag), health endpoints	Required
Accessibility	WCAG 2.1 AA for new screens	Required
Backup/DR	Daily PG backups, 14-day retention, restore drill quarterly	Required
Timezones	Store UTC; display `Asia/Kolkata` default	Required

---

13. RAG Integration Seam (Deferred)

The RAG core is **complete**. Integration happens **after** Phases 2–4. To make that cheap, define these now:

```
# ports/verification.py
class VerificationPort(Protocol):
    def verify_clauses(self, clauses: list[ClauseInput], *, anchor: str | None) -> VerificationReport: ...


class ResearchPort(Protocol):
    def ask(self, query: str, *, user_id: str, filters: dict | None) -> VerifiedAnswerDTO: ...
```

Seam	Stub behaviour now	Behaviour after integration
Draft clause verification (DR-11)	Returns `status="NOT_VERIFIED"`; UI shows neutral badge	Calls core verify endpoint; per-clause pass/quarantine badges stored in `verification_report`
Telegram `/ask`	"Coming soon" message	Routes to core query; renders answer + citation badges; fail-closed answers shown as such
Matter ↔ research linking (MT-09)	Manual link by query ID only	Auto-attach queries run inside a matter
Statutory-anchor check for deadline rules	Static anchor strings	Optional temporal check against in-force sections

Integration contract (to be honoured when wiring):

- Versioned endpoints `/api/v1/query` and `/api/v1/verify` remain stable.
- Fail-closed semantics preserved end-to-end: a quarantined clause is **never** shown as verified in drafts or Telegram.
- Latency budget for verified path stays within the 4.5 s SLA.
- Integration tests use a recorded/mocked core to keep CI fast.

---

14. Testing & Quality Strategy

Layer	Approach
Unit	Validators (CIN/DIN/PAN/GSTIN), deadline rules, number-to-words, date formatters
Golden files	Each template: fixture inputs → expected `.docx` text hash; each deadline rule: input → expected date + explanation
API / integration	pytest + test Postgres (Testcontainers); full flows for drafts, matters, calendar
Tenancy	Dedicated suite: user A cannot read/modify firm B data across every endpoint
Jobs	Scheduler tests with frozen time; idempotency of reminders
Telegram	Recorded Telegram update payloads; pairing edge cases (expiry, reuse, brute force)
Frontend	Component tests for dynamic form renderer; Playwright E2E for the three headline flows
Performance	Locust/k6 against SLAs on seeded data
Security	SAST, dependency audit, secret scanning, manual authz review
Legal review	Lawyer sign-off on templates and deadline rules before each release

CI gates: lint → unit → integration → tenancy → build → E2E smoke. Merge blocked on failure.

---

15. Risks, Open Questions & Decisions

15.1 Risks

ID	Risk	Likelihood	Impact	Mitigation
R1	Incorrect statutory deadline causes a missed filing	Med	**Critical**	Rules as reviewed data, explanations on every date, golden tests, UI disclaimer, user confirmation step
R2	Templates/rules drift from amended law	High	High	Version fields, owner + review date per rule/template, change log

ID	Risk	Likelihood	Impact	Mitigation
R3	Telegram exposes confidential data	Low	High	Pairing OTP, private chat only, short-lived links, revocation, minimal output
R4	PDF/docx rendering inconsistent across environments	Med	Med	Pin LibreOffice version in container, golden visual tests
R5	Scope creep into billing/e-filing	Med	Med	Explicit non-goals; change-control
R6	SQLite→Postgres migration surprises	Med	Med	Do in Sprint 0 with data-migration script and rollback
R7	Integration with RAG later breaks assumptions	Low	Med	Ports/adapters now; contract tests
R8	Reminder duplicates or misses	Med	High	Unique constraint on (event, user, offset, channel); idempotent worker; monitoring

15.2 Decisions (proposed — confirm)

ID	Decision	Rationale
D1	**Keep a monorepo** with clear module boundaries (`backend/app/workspace/*`) *or* split into two repos as the original PRD says. **Recommendation:** monorepo now, split later behind the ports	Existing code is a monorepo; splitting early adds CI/contract overhead with a single developer
D2	PostgreSQL + Redis + S3-compatible storage	Needed for tenancy, jobs, files
D3	Task queue: **Celery or RQ** (pick one in Sprint 0); scheduler: APScheduler/Celery beat	Reminder + export jobs
D4	Telegram library: `python-telegram-bot` (webhook mode) or `aiogram`	Mature, async
D5	PDF via LibreOffice headless from `.docx`	Single source of truth for layout
D6	Deadline rules are JSON/YAML data files, loaded + versioned	Reviewable by non-engineers
D7	Frontend forms rendered from JSON Schema–like spec	Zero-code new templates

15.3 Open questions

1. Is the target a **single advocate**, a **small firm**, or **multi-firm SaaS**? (Drives how strict tenancy/RLS must be on day one.)
2. Which **forums/benches** must be enumerated at launch?
3. Required **languages** for drafts (English only vs. Hindi/regional)?
4. Do users expect **e-mail reminders** alongside Telegram?
5. Who is the **lawyer reviewer** for templates and deadline rules?
6. Data residency requirements (India-only hosting)?

---

16. Definition of Done & Milestones

16.1 Definition of Done (per feature)

- Code merged with tests; CI green.
- Tenancy and authz tests included.
- API documented in OpenAPI; UI reviewed against design system.
- Logging/metrics added; no sensitive data in logs.
- Lawyer review completed where legal content is involved.
- Docs/README updated.

16.2 Milestones

Milestone	Contents	Exit criteria
M0 — Foundations	Green test suite, Postgres, Redis/queue, storage, tenancy base, CI	Tenancy suite passes; migrations reversible
M1 — Drafting MVP	Template registry, dynamic forms, `BOARD_RES_DIR_APPT_V1` + 3 migrated templates, docx/pdf export	§6.8 criteria met
M2 — Drafting Complete	S.241/242 petition, versioning/diff, preview, autosave, template admin	All P0/P1 DR items done
M3 — Matters	Clients, matters, documents, notes, search	MT P0 done
M4 — Calendar & Rules	Events, views, deadline engine with seed rules, reminders (in-app)	§7.5 criteria met
M5 — Dashboard	Widgets, timeline, draft↔matter linking	Dashboard SLA met
M6 — Telegram	Pairing, `/cases /case /agenda /draft_status`, reminders, digest	§8.6 criteria met
M7 — Hardening	Load tests, security review, backup drill, accessibility	NFR table satisfied
M8 — RAG Integration *(afterwards)*	Replace stubs with real adapters; `/ask`; clause verification	§13 contract tests pass

16.3 Suggested sequencing (build order)

1. M0 → 2. M1 → 3. M3 (matters first so drafts can attach) → 4. M2 → 5. M4 → 6. M5 → 7. M6 → 8. M7 → 9. M8

---

17. Appendix

A. Glossary

Term	Meaning
CIN	Corporate Identification Number (21 chars)
DIN	Director Identification Number (8 digits)
NCLT / NCLAT	National Company Law Tribunal / Appellate Tribunal
MGT-14 / DIR-12 / AOC-4 / MGT-7	MCA e-forms under the Companies Act
Port / Adapter	Interface the app depends on / concrete implementation plugged in
Fail-closed	Unverifiable content is withheld rather than shown as fact

B. Telegram sample interaction

```
User:  /auth 482913
Bot:   □ Paired with HALO as Adv. <name>. Type /agenda today to begin.

User:  /agenda today
Bot:   □ Today — Thu, 5 Nov 2026
       10:30  HEARING — <Matter A> · NCLT Mumbai · Court 3
       16:00  FILING DEADLINE — MGT-14 · <Matter B>  (due 3 Dec 2026)  [Open ►]
```

C. Dynamic form renderer — component contract

- Input: `schema.json` + current values.
- Output: controlled form with validation state, conditional sections, and a serialisable payload.
- No template-specific code allowed in components.

D. Change log vs. original PRD

- Added **Sprint 0** foundations and a **baseline/gap analysis** from the repo audit.
- Added **Clients**, tenancy columns, reminders, templates/versions, exports, Telegram link tables.
- Replaced single `reminder_sent` boolean with a **reminders** table (multi-offset, multi-channel, idempotent).
- Made deadlines a **rules engine with explanations** and a human-confirmation step.
- Defined the **RAG integration seam** and explicitly **deferred** RAG wiring.
- Added security, testing, risk register, decisions, and milestones.